

NgModules vs Standalone Components

Why Angular moved on...

Angular ≤ 16 → Angular 17–21 🚀

What most developers still use vs what Angular recommends today



ANGULAR REALITY CHECK

- Angular is at v21
- Most production apps still use NgModules

Why?

- 👉 Stability
- 👉 Large legacy codebases
- 👉 Migration cost

Framework evolves faster than real projects



NGMODULES (TRADITIONAL ANGULAR)

- Core architecture till Angular v15
- Acts as a container

Manages:

- Components
- Directives
- Pipes
- Services
- Imports & Exports

 Module-first architecture



NGMODULE-BASED APP

```
@NgModule({  
  declarations: [ProductComponent],  
  imports: [CommonModule],  
})  
export class ProductModule {}
```

- 😞 Multiple files
- 😞 Declarations & exports
- 😞 Boilerplate everywhere



WHY DEVS GOT TIRED

- ✗ Too much boilerplate
- ✗ Hard to trace dependencies
- ✗ Confusing for beginners
- ✗ Overkill for small features
- ✗ Complex lazy loading

Powerful... but heavy



ANGULAR ASKED A QUESTION

“Why can’t a component manage its own dependencies?”

💡 Answer → Standalone Components

Angular v14+

Standalone-first from v17+



STANDALONE COMPONENT

Standalone Component

- No NgModule required
- Component owns its imports
- Can be lazy-loaded directly
- Cleaner & modern approach

🧩 Component-first architecture

```
@Component({
  standalone: true,
  imports: [CommonModule],
  template: `<h2>Products</h2>`
})
export class ProductComponent {}
```

- ✓ No module
- ✓ Less code
- ✓ Easy to understand



LAZY LOADING COMPARISON

NgModule ❌

```
loadChildren: () => import('./product.module')
```

Standalone ✅

```
loadComponent: () => import('./product.component')
```



WHY ANGULAR RECOMMENDS IT

- ✓ Less boilerplate
- ✓ Easier lazy loading
- ✓ Better readability
- ✓ Faster onboarding
- ✓ Future-proof architecture

Modern Angular = Standalone



WHY NGMODULES STILL EXIST

- ✗ Large apps already use them
- ✗ Migration cost is high
- ✗ Some libraries still expect modules
- ✗ Teams are used to old patterns

Enterprise reality matters



DECISION GUIDE

- New app →  Standalone
- Small / Medium app →  Standalone
- Learning Angular →  Standalone
- Large existing app →  Keep NgModules
- Migration →  Hybrid approach



THE TRUTH

- NgModules are not dead
- Standalone is the future
- Angular supports both
- Choose based on project reality



ARE YOU STILL USING

NGMODULES?

👉 Comment your project version

❤️ Like | 🔄 Share | 💬 Discuss

